```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report)
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.layers import Input, Dense

df = pd.read_csv("seeds_new (1).csv")

df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 210,\n  \"fields\": [\
n    {\n      \"column\": \"Area\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 2.909699430687361,\n
\"min\": 10.59,\n        \"max\": 21.18,\n
\"num_unique_values\": 193,\n        \"samples\": [\n          14.86,\
n          12.49,\n          16.53\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Perimeter\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.305958726564022,\n        \"min\": 12.41,\n        \"max\": 17.25,\n
\"num_unique_values\": 170,\n        \"samples\": [\n          13.45,\
n          14.41,\n          13.92\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Compactness\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.0236294165838465,\n        \"min\": 0.8081,\n        \"max\":
0.9183,\n        \"num_unique_values\": 186,\n        \"samples\": [\n
0.9064,\n          0.856,\n          0.8372\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Length of kernel\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.4430634777264492,\n        \"min\": 4.899,\n        \"max\": 6.675,\
n        \"num_unique_values\": 188,\n        \"samples\": [\n
5.183,\n          5.186,\n          5.205\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Width of kernel\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
```

```
0.37771444490658723,\n        \"min\": 2.63,\n        \"max\": 4.033,\
n        \"num_unique_values\": 184,\n        \"samples\": [\n
3.168,\n          3.393,\n          2.641\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Asymmetry coefficient\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.5035571308217794,\n        \"min\": 0.7651,\n        \"max\":
8.456,\n        \"num_unique_values\": 207,\n        \"samples\": [\n
6.388,\n          4.185,\n          4.92\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Length of kernel groove\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.4914804991024054,\n        \"min\": 4.519,\n        \"max\": 6.55,\n
\"num_unique_values\": 148,\n        \"samples\": [\n          5.308,\
n          4.703,\n          4.957\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Class (1, 2, 3)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0,\n        \"min\": 1,\n        \"max\": 3,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          1,\n
2,\n          3\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Unnamed: 8\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 1,\n        \"samples\":
[\n          \" \"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Unnamed: 9\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 1,\n        \"samples\":
[\n          \" \"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```


```
df.tail()
```


```
{"repr_error":"0","type":"dataframe"}
```


```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 210 entries, 0 to 209
Data columns (total 10 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Area                    210 non-null    float64
 1   Perimeter               210 non-null    float64
 2   Compactness             210 non-null    float64
 3   Length of kernel        210 non-null    float64
 4   Width of kernel         210 non-null    float64
 5   Asymmetry coefficient   210 non-null    float64
 6   Length of kernel groove  210 non-null    float64
```

```
 7   Class (1, 2, 3)          210 non-null    int64
 8   Unnamed: 8               2 non-null      object
 9   Unnamed: 9               1 non-null      object
dtypes: float64(7), int64(1), object(2)
memory usage: 16.5+ KB

df.describe()

{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n
{\n      \"column\": \"Area\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 69.7290183208939,\n
\"min\": 2.909699430687361,\n        \"max\": 210.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
14.84752380952381,\n          14.355,\n          210.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Perimeter\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
69.92065326255143,\n        \"min\": 1.305958726564022,\n
\"max\": 210.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          14.559285714285714,\n          14.32,\n
210.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Compactness\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 73.98219304279458,\n        \"min\":
0.0236294165838465,\n        \"max\": 210.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
0.8709985714285714,\n          0.8734500000000001,\n          210.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Length of kernel\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
72.53303899953512,\n        \"min\": 0.4430634777264492,\n
\"max\": 210.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          5.628533333333334,\n          5.5235,\n
210.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Width of kernel\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 73.24211837620656,\n        \"min\":
0.37771444490658723,\n        \"max\": 210.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
3.258604761904762,\n          3.237,\n          210.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Asymmetry coefficient\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
73.00327979031506,\n        \"min\": 0.7651,\n        \"max\": 210.0,\
n        \"num_unique_values\": 8,\n        \"samples\": [\n
3.700200952380953,\n          3.599,\n          210.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Length of kernel groove\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
72.59679119462936,\n        \"min\": 0.4914804991024054,\n
```

```
\"max\": 210.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          5.408071428571429,\n
5.223000000000001,\n          210.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Class (1, 2, 3)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
73.60382587406471,\n        \"min\": 0.8184475910711349,\n
\"max\": 210.0,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          2.0,\n          3.0,\n
0.8184475910711349\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```

```
df.isnull().sum()
```

```
Area                          0
Perimeter                     0
Compactness                   0
Length of kernel              0
Width of kernel               0
Asymmetry coefficient         0
Length of kernel groove       0
Class (1, 2, 3)               0
Unnamed: 8                  208
Unnamed: 9                  209
dtype: int64
```

```
df.shape
```

```
(210, 10)
```

```
X=df.drop("Class (1, 2, 3)",axis=1)
y_original=df["Class (1, 2, 3)"]
print("Input shape",X.shape)
print("Target shape",y_original.shape)
```

```
Input shape (210, 9)
Target shape (210,)
```

```
y=y_original-1
print("original classes:")
print(sorted(y_original.unique()))
print("\nANN encoded classes:")
print(sorted(y.unique()))
```

```
original classes:
[np.int64(1), np.int64(2), np.int64(3)]

ANN encoded classes:
[np.int64(0), np.int64(1), np.int64(2)]
```

```
X_train ,X_test,y_train,y_test=train_test_split(X,y,test_size=0.20,ran
dom_state=42,stratify=y)
print("training samples:",X_train.shape[0])
print("testing samples:",X_test.shape[0])
```

```
training samples: 168
testing samples: 42
```

```
X_cleaned = X.drop(columns=["Unnamed: 8", "Unnamed: 9"],
errors="ignore")


X_train, X_test, y_train, y_test = train_test_split(X_cleaned, y,
test_size=0.20, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Data cleaned and scaled successfully!")
print("Features shape after removing unneeded columns:",
X_train_scaled.shape)
```

```
Data cleaned and scaled successfully!
Features shape after removing unneeded columns: (168, 7)
```

```
scaler=StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)

scaler.fit_transform(X_test)
```

```
array([[-0.13481356, -0.30885661,  1.63834475, -0.72108928,
0.28326462,
        -0.76430253, -1.20943602],
       [-0.86696115, -0.79265998, -1.079508  , -0.44412568, -
1.01611436,
        -0.08296069, -0.1493714 ],
       [ 0.3031065 ,  0.49244271, -0.70339658,  0.66153059,
0.00779627,
        -0.80711315,  0.9685525 ],
       [-1.32882996, -1.4578896 , -0.42131302, -1.40250766, -
1.05769449,
         0.74059948, -0.71763216],
       [ 1.23710786,  1.05184035,  1.89804073,  0.79121989,
1.36954545,
        -0.66416923,  1.14006393],
       [-0.46667485, -0.42224803, -0.31833013, -0.17155833, -
0.40280748,
         2.38191499,  0.06140168],
       [ 2.15058173,  2.0799225 ,  0.78313902,  2.07492419,
```

```
1.98285233,
         0.67094153,  1.87157051],
       [ 0.37495276,  0.31101645,  1.0697001 ,  0.12079214,
0.53534414,
         1.33559449, -0.56678476],
       [ 1.65107916,  1.67171341,  0.42941519,  1.6418938 ,
1.48908831,
        -0.57346877,  1.60087175],
       [ 1.28842662,  1.22570719,  1.06074507,  1.07257974,
1.40592806,
        -1.10098264,  0.79290826],
       [-0.8909099 , -0.95896738, -0.08997677, -1.14312905, -
0.75883733,
        -1.69597765, -0.92840524],
       [-0.85327615, -0.86069483, -0.4705657 , -0.83319359, -
0.82900379,
         0.80155019, -0.5213239 ],
       [ 0.98051408,  0.87797352,  1.37417125,  0.37797262,
1.20062618,
         1.17088246,  0.51807669],
       [ 1.45606789,  1.37689574,  1.15477292,  1.15391032,  1.5514585
,
        -0.61265137,  1.11320069],
       [-1.07223618, -1.05723994, -1.0257778 , -0.99365663, -0.9953243
,
         2.14972181, -0.93873726],
       [-0.42219859, -0.36177261, -0.35862778, -0.22651142, -
0.42359755,
        -0.16060028, -0.64737447],
       [-1.14066118, -1.04212109, -1.76009056, -0.97387351, -
1.45530446,
         0.42423628, -0.57505037],
       [-0.32298233, -0.33909432,  0.26822458, -0.30124763, -
0.29625841,
        -0.59523688, -1.20943602],
       [-1.2843537 , -1.28402277, -1.31681639, -1.25962961, -
1.32796532,
         1.19410178, -0.75482741],
       [ 0.27915774,  0.27321931,  0.60851587,  0.56701127,
0.43659134,
         0.69561205,  0.24324513],
       [-0.06638855,  0.08423362, -0.65414389,  0.21091521, -
0.29885717,
         0.26387787, -0.02952004],
       [-0.76090239, -0.82289769,  0.09807894, -0.84198609, -
0.54833793,
         0.89587867, -0.54405433],
       [ 0.50838152,  0.66630954, -0.43026805,  0.31422703,
0.41320252,
```

```
         0.38723049,  0.60486561],
       [-1.11671243, -1.2386662 , -0.00490395, -1.34315831, -
0.98233051,
        -0.76357693, -1.65991182],
       [ 1.25763537,  1.52808429, -1.03921035,  2.30133094,  0.6678808
,
         0.85669607,  2.14433567],
       [-0.19639607, -0.18034634,  0.16971921, -0.21771893, -
0.15592548,
        -0.77228418, -0.39527333],
       [-1.18171619, -1.41253304,  0.6980662 , -1.57615944, -
0.90696653,
        -1.07631211, -1.46153716],
       [-0.61720987, -0.75486284,  0.87268936, -1.0552041 , -
0.28326462,
        -1.01608701, -1.65991182],
       [-0.90117365, -0.82289769, -1.20040095, -0.77384425, -
0.99012678,
         0.89660427, -0.75276101],
       [-0.17928981, -0.07451436, -0.44817812,  0.08562216, -
0.25727704,
        -0.49365237, -0.48412865],
       [-1.13723993, -1.09503708, -1.35263653, -0.97607164, -
1.22921252,
         0.21453682, -0.30435161],
       [ 1.19263161,  1.33153917, -0.21086972,  1.43307204,
0.73804726,
        -0.65255957,  1.78271519],
       [ 0.06704022,  0.05399591,  0.56821821, -0.14298272,
0.27286959,
        -1.98404231, -0.93873726],
       [-1.11329118, -1.03456166, -1.56307982, -0.97387351, -1.3227678
,
        -1.11114109, -0.57505037],
       [-0.8293274 , -0.67926856, -1.57203485, -0.46390879, -1.0265094
,
         0.72826422, -0.10391054],
       [ 1.32263912,  1.30130146,  0.81000412,  1.17369344,
1.15904605,
        -1.46160766,  0.99954854],
       [ 0.74786905,  0.87041409, -0.16161704,  1.1429197 ,
0.61070812,
        -0.04087567,  1.15039594],
       [ 1.33632413,  1.17279119,  1.70998502,  0.91871107,
1.64241503,
         1.63164079,  0.96441969],
       [-0.91143741, -0.88337311, -0.81085699, -0.59139997, -
1.08628083,
         0.37634644, -0.21136348],
```

```
       [ 0.81971531,  0.66630954,  1.76819274,  0.4680957 ,
1.18243487,
        -0.57782239,  0.1523234 ],
       [-0.70616238, -0.62635257, -0.90936236, -0.45951255, -
0.82380627,
        -0.32386111, -0.3704765 ],
       [ 1.52791415,  1.4751683 ,  0.92194205,  1.37592082,
1.52547093,
        -0.48494512,  1.60087175]])

model=Sequential([
    Input(shape=(7,)),
    Dense(16,activation="relu"),
    Dense(8,activation="relu"),
    Dense(3,activation="softmax")
])

model.summary()
```

```
Model: "sequential"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
┩
│ dense (Dense)                        │ (None, 16)                  │
128 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dense_1 (Dense)                      │ (None, 8)                   │
136 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dense_2 (Dense)                      │ (None, 3)                   │
27 │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────
┘
```

```
 Total params: 291 (1.14 KB)

 Trainable params: 291 (1.14 KB)

 Non-trainable params: 0 (0.00 B)
```

```
model.compile(
    optimizer =Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
loss="sparse_categorical_crossentropy"

early_stop=EarlyStopping(
    monitor="val_loss",
    patience=10,
    restore_best_weights=True
)

history=model.fit(
    X_train_scaled,
    y_train,
    epochs=100,
    batch_size=16,
    validation_split=0.20,
    callbacks=[early_stop],
    verbose=1
)

Epoch 1/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 4s 80ms/step - accuracy: 0.6567 - loss:
0.8935 - val_accuracy: 0.4706 - val_loss: 0.8968
Epoch 2/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 1s 28ms/step - accuracy: 0.6642 - loss:
0.8344 - val_accuracy: 0.4706 - val_loss: 0.8457
Epoch 3/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.6567 - loss:
0.7852 - val_accuracy: 0.4706 - val_loss: 0.8030
Epoch 4/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step - accuracy: 0.6567 - loss:
0.7428 - val_accuracy: 0.4706 - val_loss: 0.7680
Epoch 5/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step - accuracy: 0.6642 - loss:
0.7038 - val_accuracy: 0.5000 - val_loss: 0.7368
Epoch 6/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.6642 - loss:
0.6682 - val_accuracy: 0.5294 - val_loss: 0.7091
Epoch 7/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.6642 - loss:
0.6359 - val_accuracy: 0.5294 - val_loss: 0.6789
Epoch 8/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - accuracy: 0.6642 - loss:
0.6044 - val_accuracy: 0.5294 - val_loss: 0.6545
Epoch 9/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.6716 - loss:
0.5754 - val_accuracy: 0.5294 - val_loss: 0.6271
Epoch 10/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.7015 - loss:
0.5457 - val_accuracy: 0.6176 - val_loss: 0.5902
Epoch 11/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.7687 - loss:
```

```
0.5110 - val_accuracy: 0.8529 - val_loss: 0.5384
Epoch 12/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.8657 - loss:
0.4702 - val_accuracy: 0.8824 - val_loss: 0.4846
Epoch 13/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step - accuracy: 0.9030 - loss:
0.4253 - val_accuracy: 0.9412 - val_loss: 0.4365
Epoch 14/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.8955 - loss:
0.3913 - val_accuracy: 0.9412 - val_loss: 0.3925
Epoch 15/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.9104 - loss:
0.3593 - val_accuracy: 0.9412 - val_loss: 0.3585
Epoch 16/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9104 - loss:
0.3350 - val_accuracy: 0.9412 - val_loss: 0.3288
Epoch 17/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.9104 - loss:
0.3140 - val_accuracy: 0.9412 - val_loss: 0.3080
Epoch 18/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9179 - loss:
0.2964 - val_accuracy: 0.9412 - val_loss: 0.2929
Epoch 19/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9104 - loss:
0.2820 - val_accuracy: 0.9412 - val_loss: 0.2788
Epoch 20/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.9104 - loss:
0.2698 - val_accuracy: 0.9412 - val_loss: 0.2632
Epoch 21/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9104 - loss: 0.2583
- val_accuracy: 0.9412 - val_loss: 0.2528
Epoch 22/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9104 - loss: 0.2481
- val_accuracy: 0.9412 - val_loss: 0.2422
Epoch 23/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9179 - loss:
0.2400 - val_accuracy: 0.9412 - val_loss: 0.2338
Epoch 24/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9179 - loss: 0.2312
- val_accuracy: 0.9412 - val_loss: 0.2255
Epoch 25/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9179 - loss: 0.2242
- val_accuracy: 0.9412 - val_loss: 0.2198
Epoch 26/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9104 - loss: 0.2165
- val_accuracy: 0.9412 - val_loss: 0.2140
Epoch 27/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9179 - loss: 0.2099
- val_accuracy: 0.9412 - val_loss: 0.2108
```

```
Epoch 28/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9254 - loss:
0.2035 - val_accuracy: 0.9412 - val_loss: 0.2056
Epoch 29/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9254 - loss: 0.1978
- val_accuracy: 0.9412 - val_loss: 0.1979
Epoch 30/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9254 - loss: 0.1935
- val_accuracy: 0.9412 - val_loss: 0.1890
Epoch 31/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9254 - loss: 0.1886
- val_accuracy: 0.9412 - val_loss: 0.1847
Epoch 32/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9254 - loss: 0.1838
- val_accuracy: 0.9412 - val_loss: 0.1824
Epoch 33/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9254 - loss: 0.1799
- val_accuracy: 0.9412 - val_loss: 0.1821
Epoch 34/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9179 - loss: 0.1766
- val_accuracy: 0.9412 - val_loss: 0.1808
Epoch 35/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9254 - loss: 0.1726
- val_accuracy: 0.9412 - val_loss: 0.1782
Epoch 36/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9328 - loss: 0.1686
- val_accuracy: 0.9412 - val_loss: 0.1756
Epoch 37/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9328 - loss: 0.1656
- val_accuracy: 0.9412 - val_loss: 0.1739
Epoch 38/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9403 - loss:
0.1619 - val_accuracy: 0.9412 - val_loss: 0.1695
Epoch 39/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9403 - loss: 0.1586
- val_accuracy: 0.9412 - val_loss: 0.1665
Epoch 40/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9478 - loss: 0.1553
- val_accuracy: 0.9412 - val_loss: 0.1633
Epoch 41/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9403 - loss: 0.1527
- val_accuracy: 0.9412 - val_loss: 0.1601
Epoch 42/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9403 - loss: 0.1494
- val_accuracy: 0.9412 - val_loss: 0.1568
Epoch 43/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9403 - loss:
0.1471 - val_accuracy: 0.9412 - val_loss: 0.1544
Epoch 44/100
```

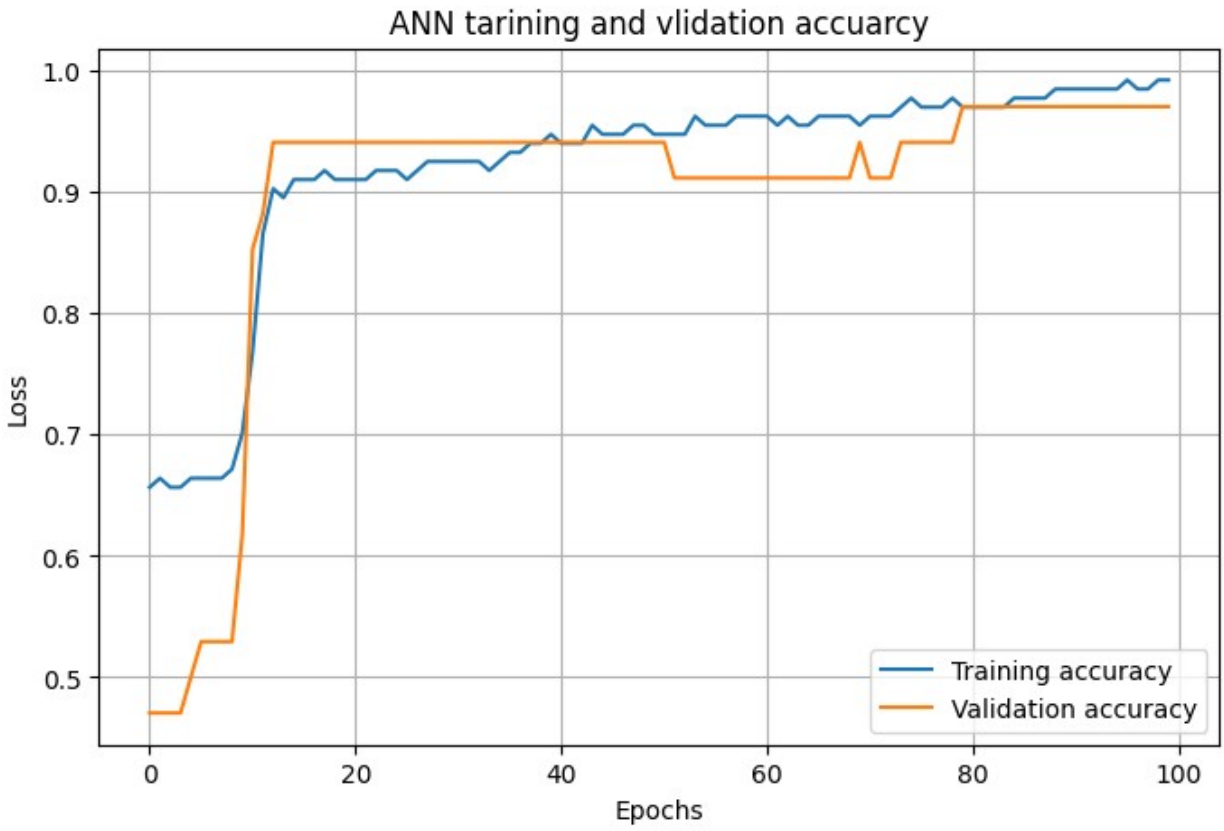
```
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9552 - loss:
0.1439 - val_accuracy: 0.9412 - val_loss: 0.1524
Epoch 45/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9478 - loss:
0.1410 - val_accuracy: 0.9412 - val_loss: 0.1526
Epoch 46/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9478 - loss:
0.1384 - val_accuracy: 0.9412 - val_loss: 0.1500
Epoch 47/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9478 - loss:
0.1355 - val_accuracy: 0.9412 - val_loss: 0.1477
Epoch 48/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9552 - loss:
0.1326 - val_accuracy: 0.9412 - val_loss: 0.1436
Epoch 49/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9552 - loss:
0.1296 - val_accuracy: 0.9412 - val_loss: 0.1401
Epoch 50/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9478 - loss:
0.1276 - val_accuracy: 0.9412 - val_loss: 0.1367
Epoch 51/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9478 - loss:
0.1255 - val_accuracy: 0.9412 - val_loss: 0.1355
Epoch 52/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9478 - loss:
0.1218 - val_accuracy: 0.9118 - val_loss: 0.1337
Epoch 53/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9478 - loss:
0.1186 - val_accuracy: 0.9118 - val_loss: 0.1324
Epoch 54/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9627 - loss:
0.1156 - val_accuracy: 0.9118 - val_loss: 0.1290
Epoch 55/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9552 - loss:
0.1127 - val_accuracy: 0.9118 - val_loss: 0.1269
Epoch 56/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9552 - loss:
0.1100 - val_accuracy: 0.9118 - val_loss: 0.1258
Epoch 57/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9552 - loss:
0.1076 - val_accuracy: 0.9118 - val_loss: 0.1243
Epoch 58/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9627 - loss:
0.1054 - val_accuracy: 0.9118 - val_loss: 0.1219
Epoch 59/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9627 - loss: 0.1029
- val_accuracy: 0.9118 - val_loss: 0.1209
Epoch 60/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9627 - loss: 0.1013
```

```
- val_accuracy: 0.9118 - val_loss: 0.1204
Epoch 61/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9627 - loss: 0.0992
- val_accuracy: 0.9118 - val_loss: 0.1187
Epoch 62/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9552 - loss: 0.0986
- val_accuracy: 0.9118 - val_loss: 0.1163
Epoch 63/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9627 - loss: 0.0962
- val_accuracy: 0.9118 - val_loss: 0.1155
Epoch 64/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9552 - loss: 0.0943
- val_accuracy: 0.9118 - val_loss: 0.1160
Epoch 65/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9552 - loss: 0.0922
- val_accuracy: 0.9118 - val_loss: 0.1131
Epoch 66/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9627 - loss: 0.0908
- val_accuracy: 0.9118 - val_loss: 0.1100
Epoch 67/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9627 - loss: 0.0893
- val_accuracy: 0.9118 - val_loss: 0.1087
Epoch 68/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9627 - loss:
0.0881 - val_accuracy: 0.9118 - val_loss: 0.1077
Epoch 69/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9627 - loss: 0.0868
- val_accuracy: 0.9118 - val_loss: 0.1078
Epoch 70/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9552 - loss:
0.0861 - val_accuracy: 0.9412 - val_loss: 0.1086
Epoch 71/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9627 - loss: 0.0841
- val_accuracy: 0.9118 - val_loss: 0.1064
Epoch 72/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9627 - loss: 0.0822
- val_accuracy: 0.9118 - val_loss: 0.1051
Epoch 73/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9627 - loss: 0.0824
- val_accuracy: 0.9118 - val_loss: 0.1084
Epoch 74/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9701 - loss: 0.0798
- val_accuracy: 0.9412 - val_loss: 0.1026
Epoch 75/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9776 - loss: 0.0780
- val_accuracy: 0.9412 - val_loss: 0.1003
Epoch 76/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9701 - loss: 0.0774
- val_accuracy: 0.9412 - val_loss: 0.0979
```

```
Epoch 77/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9701 - loss: 0.0754
- val_accuracy: 0.9412 - val_loss: 0.0982
Epoch 78/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9701 - loss:
0.0749 - val_accuracy: 0.9412 - val_loss: 0.0996
Epoch 79/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9776 - loss: 0.0731
- val_accuracy: 0.9412 - val_loss: 0.0950
Epoch 80/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9701 - loss: 0.0723
- val_accuracy: 0.9706 - val_loss: 0.0917
Epoch 81/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9701 - loss: 0.0709
- val_accuracy: 0.9706 - val_loss: 0.0911
Epoch 82/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9701 - loss: 0.0696
- val_accuracy: 0.9706 - val_loss: 0.0906
Epoch 83/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9701 - loss: 0.0696
- val_accuracy: 0.9706 - val_loss: 0.0874
Epoch 84/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9701 - loss: 0.0671
- val_accuracy: 0.9706 - val_loss: 0.0875
Epoch 85/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9776 - loss: 0.0662
- val_accuracy: 0.9706 - val_loss: 0.0869
Epoch 86/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9776 - loss: 0.0650
- val_accuracy: 0.9706 - val_loss: 0.0837
Epoch 87/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9776 - loss: 0.0637
- val_accuracy: 0.9706 - val_loss: 0.0847
Epoch 88/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9776 - loss:
0.0632 - val_accuracy: 0.9706 - val_loss: 0.0834
Epoch 89/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9851 - loss: 0.0622
- val_accuracy: 0.9706 - val_loss: 0.0810
Epoch 90/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9851 - loss: 0.0613
- val_accuracy: 0.9706 - val_loss: 0.0806
Epoch 91/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9851 - loss: 0.0604
- val_accuracy: 0.9706 - val_loss: 0.0789
Epoch 92/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9851 - loss: 0.0597
- val_accuracy: 0.9706 - val_loss: 0.0769
Epoch 93/100
```

```
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9851 - loss: 0.0590
- val_accuracy: 0.9706 - val_loss: 0.0765
Epoch 94/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9851 - loss:
0.0580 - val_accuracy: 0.9706 - val_loss: 0.0762
Epoch 95/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9851 - loss: 0.0565
- val_accuracy: 0.9706 - val_loss: 0.0767
Epoch 96/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9925 - loss: 0.0561
- val_accuracy: 0.9706 - val_loss: 0.0805
Epoch 97/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9851 - loss: 0.0553
- val_accuracy: 0.9706 - val_loss: 0.0788
Epoch 98/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9851 - loss: 0.0543
- val_accuracy: 0.9706 - val_loss: 0.0744
Epoch 99/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9925 - loss:
0.0526 - val_accuracy: 0.9706 - val_loss: 0.0730
Epoch 100/100
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9925 - loss: 0.0515
- val_accuracy: 0.9706 - val_loss: 0.0721
```

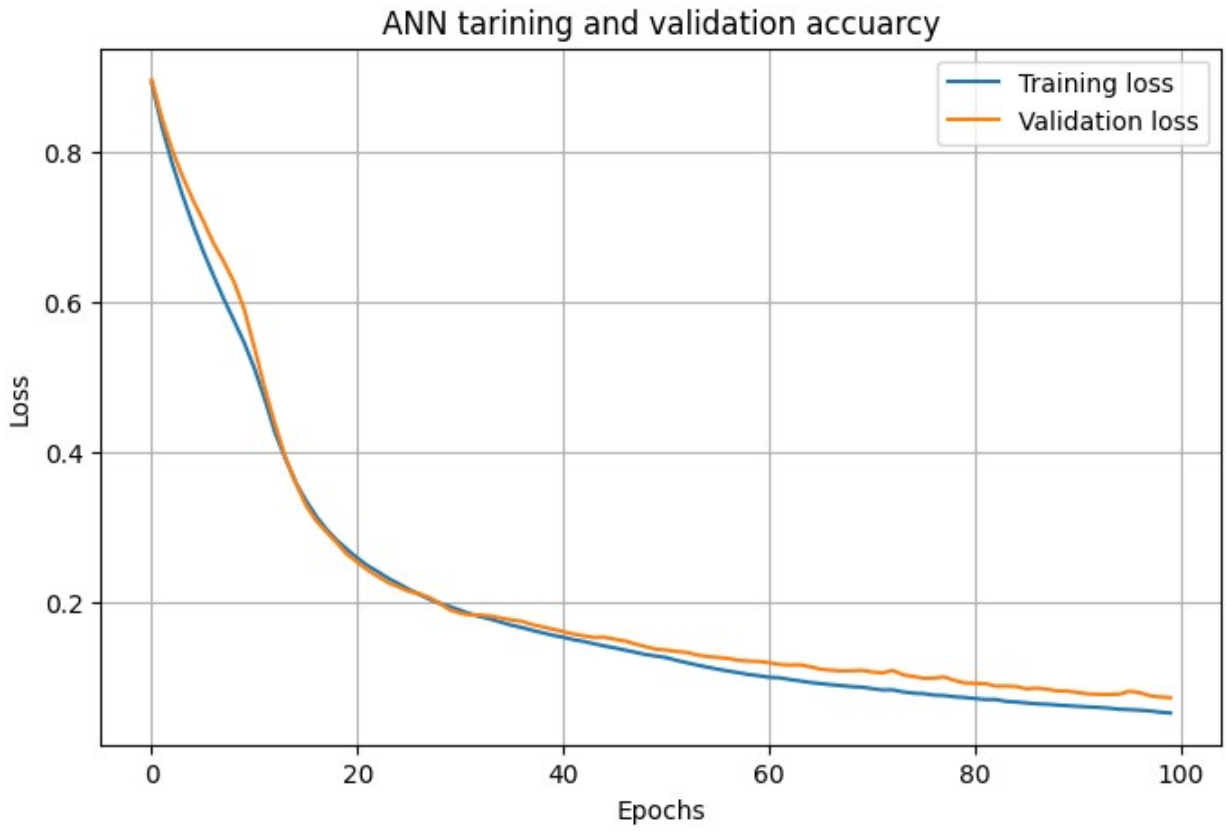
```
plt.figure(figsize=(8,5))
plt.plot(history.history["accuracy"],label="Training accuracy")
plt.plot(history.history["val_accuracy"],label="Validation accuracy")
plt.title("ANN tarining and vlidation accuarcy")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```




```
print("Training completed")
print("Number of epochs actually
completed:",len(history.history["loss"]))
```

```
Training completed
Number of epochs actually completed: 100
```

```
plt.figure(figsize=(8,5))
plt.plot(history.history["loss"],label="Training loss")
plt.plot(history.history["val_loss"],label="Validation loss")
plt.title("ANN tarining and validation accuarcy")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```




```
test_loss,test_accuracy=model.evaluate(X_test_scaled,y_test,verbose=1)
print("Test loss:",test_loss)
print("Test accuracy:",test_accuracy)
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - accuracy: 0.8333 - loss:
0.3604
Test loss: 0.3604251742362976
Test accuracy: 0.8333333134651184
```

```
y_prob=model.predict(X_test_scaled,verbose=1)
print(y_prob[:5])
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 130ms/step
[[9.9866784e-01 4.3729640e-04 8.9487690e-04]
 [2.6608031e-02 2.0254999e-04 9.7318947e-01]
 [5.1393491e-01 4.7524342e-01 1.0821622e-02]
 [1.3055329e-03 1.2764279e-05 9.9868178e-01]
 [1.5304451e-03 9.9846917e-01 4.0168501e-07]]
```

```
y_pred=np.argmax(y_prob,axis=1)
print("predicted encoded classes:")
print(y_pred[:10])
```

```
predicted encoded classes:
[0 2 0 2 1 2 1 1 1 1]
```

```
y_pred_original=y_pred+1
y_test_original=y_test+1
print("predicted classes:")
print(y_pred_original[:10])
```

```
predicted classes:
[1 3 1 3 2 3 2 2 2 2]
```

```
comparison=pd.DataFrame({
    "actual":y_test_original.values,
    "predicted":y_pred_original
})
comparison.head(15)
```


```
{"summary":"{\n  \"name\": \"comparison\",\n  \"rows\": 42,\n
\"fields\": [\n    {\n      \"column\": \"actual\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0,\n        \"min\": 1,\n        \"max\": 3,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          1,\n
3,\n          2\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"predicted\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0,\n        \"min\": 1,\n
\"max\": 3,\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          1,\n          3,\n          2\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"comparison"}
```


```
accuracy= accuracy_score(y_test_original,y_pred_original)
print("Accuracy: ",accuracy)
```

```
Accuracy:  0.8333333333333334
```

```
precision=precision_score(y_test_original,y_pred_original,average='wei
ghted')
print("precision:",precision)
```

```
precision: 0.8416394335511983
```

```
recall=recall_score(y_test_original,y_pred_original,average='weighted'
)
print("recall:",recall)
```

```
recall: 0.8333333333333334
```

```
f1=f1_score(y_test_original,y_pred_original,average='weighted')
print("f1:",f1)
```

```
f1: 0.8218482156771078
```

```
cm=confusion_matrix(y_test_original,y_pred_original)
print(cm)
```

```
[[ 8  3  3]
 [ 1 13  0]
 [ 0  0 14]]
```